

Lab 05 Template

Part 01

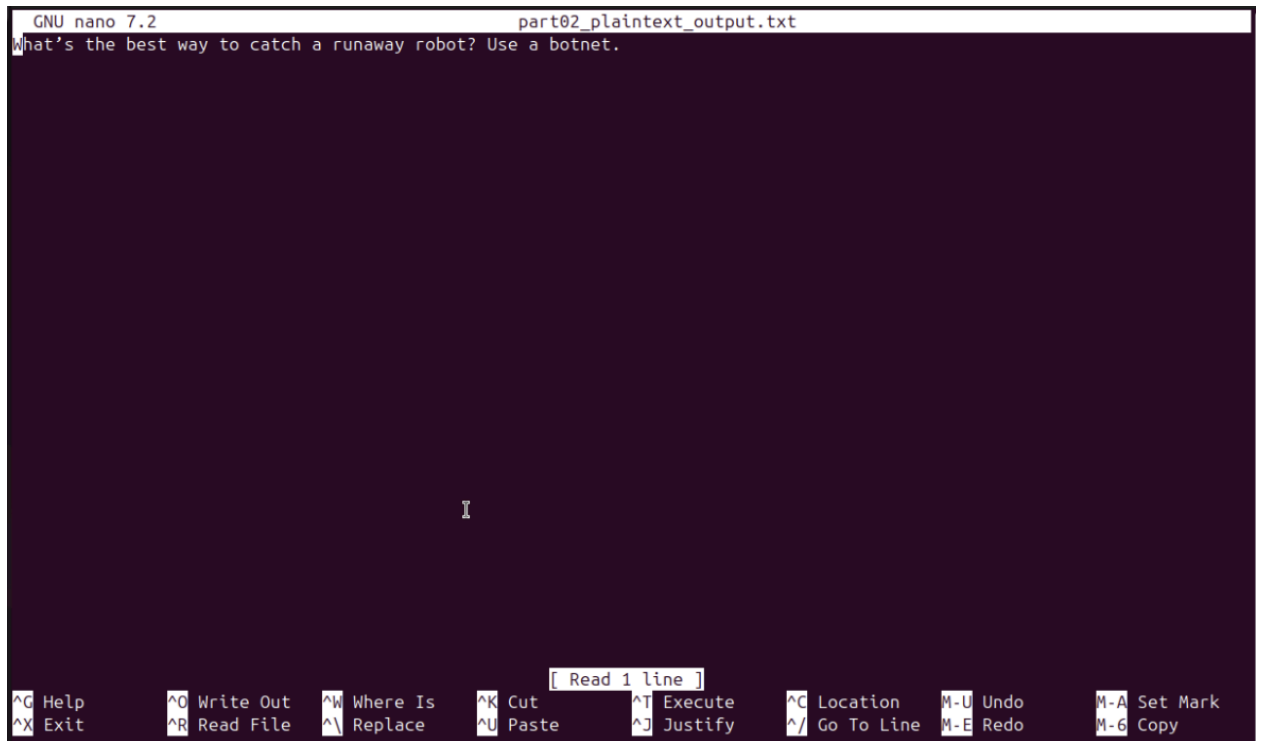
- 1) Print the part01_ciphertext.bin file and include a screenshot of the result in your lab report.
(15 points)

```
cpre3310@cpre3310:~/lab05/part01$ xxd -b part01_ciphertext.bin
00000000: 01011111 01011101 10001110 10111100 01001000 10111011  _]..H.
00000006: 00010011 01110010 11010111 10001011 00001011 10011111  .r....
0000000c: 11100111 01100010 11011010 00011010 00111001 11110111  .b..9.
00000012: 10000100 00010101 01000000 00011011 10110001 01110110  ..@..v
00000018: 00111001 00000110 00100100 10110010 00100000 10100000  9.$. .
0000001e: 01101010 00010000 11001101 01011000 01010110 01000100  j..XVD
00000024: 11001001 01011011 10000000 10011110 11100010 11000001  .[....
0000002a: 10010010 01001011 01100011 10001010 00010000 10100110  .Kc...
00000030: 01100000 01101010 10100000 11000100 00010100 10010000  `j....
00000036: 10100011 01011000 11011011 11011111 00001000 11001011  .X....
0000003c: 11110000 01011100 01000110 11000011 11001000 10000010  .\F...
00000042: 00000000 00100111  .'
cpre3310@cpre3310:~/lab05/part01$
```

Part 02

- 2) Include a screenshot in your lab report of the resulting ASCII text printed on your screen when you decrypted from part02_ciphertext.bin.
(15 points)

```
cpre3310@cpre3310:~/lab05/part02$ python3 ../part01/part01_02_encrypt_decrypt_chacha_skel.py decrypt part02_ciphertext.b
in part02_plaintext_output.txt part02_key.bin part02_nonce.bin
File 'part02_ciphertext.bin' decrypted successfully to 'part02_plaintext_output.txt'.
Warning: The decrypted content contains non-ASCII characters and could not be fully decoded as ASCII.
cpre3310@cpre3310:~/lab05/part02$ cat part02_plaintext_output.txt
What's the best way to catch a runaway robot? Use a botnet. cpre3310@cpre3310:~/lab05/part02$ cat part02_plaintext_outpu
```



The screenshot shows a terminal window with the following content:

```
GNU nano 7.2 part02_plaintext_output.txt
What's the best way to catch a runaway robot? Use a botnet.
```

The terminal window is running GNU nano 7.2 editor. The status bar at the bottom shows various keyboard shortcuts: ^G Help, ^O Write Out, ^W Where Is, ^K Cut, ^T Execute, ^C Location, M-U Undo, M-A Set Mark, ^X Exit, ^R Read File, ^_ Replace, ^U Paste, ^J Justify, ^/ Go To Line, M-E Redo, M-6 Copy. A tooltip "[Read 1 line]" is visible over the ^U Paste shortcut.

- 3) Upload your working <netid> part01_02_encrypt_decrypt_chacha_skel.py code as a separate file.
(20 points)

Part 03

- 4) Include a screenshot in your lab report of the resulting ASCII text printed on your screen, which is part03_plaintext2.txt printed on your screen.
(15 points)

```
cpre3310@cpre3310:~/lab05/part03$ python3 part03_xor_nonce_reuse_skel.py part03_ciphertext1.bin part03_ciphertext2.bin p
art03_plaintext1.txt Part03_recovered_plaintext2.txt
Recovered Plaintext 2 (ASCII): Why did the password lack confidence? It was insecure.
Recovered plaintext written to 'Part03_recovered_plaintext2.txt'.
```

```
GNU nano 7.2 Part03_recovered_plaintext2.txt
Why did the password lack confidence? It was insecure.

[ Read 1 line ]
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^I Execute    ^C Location   ^U Undo       ^A Set Mark
^X Exit      ^R Read File  ^V Replace    ^U Paste      ^J Justify    ^_ Go To Line  ^E Redo       ^B Copy
```

- 5) Questions related to reusing nonce, but not the key
- a. **Would a unique keystream be generated if the nonce was reused, but the key changed?**
(5 points)

Ans) Over here, we can say that a unique keystream will be generated if the key changed and even if the nonce is reused. So, ChaCha20 generates its keystream using a combination of the secret key, the nonce, and the counter. Also, since a key is a core input to the keystream generation function, changing the key results in a completely different keystream.

- b. **Would you really want to implement ChaCha20 with the nonce reused? Why or why not?**
(10 points)

Ans) In my opinion, we should never reuse a nonce with the same key in ChaCha 20 since it causes the exact keystream to be generated and as Chacha 20 is a stream cipher, where encryption works as $\text{Ciphertext} = \text{Plaintext} \oplus \text{keyStream}$, so if 2 plaintexts are encrypted together, with the same key and nonce, we will get:

$$\begin{aligned} C1 &= P1 \oplus K \\ C2 &= P2 \oplus K \end{aligned}$$

So, an attacker can basically do the following, he can basically get rid of the keystream completely and if he knows one of the texts then he can easily recover information about the other plaintext as well, which completely breaks confidentiality.

$$C1 \oplus C2 = P1 \oplus P2$$

- 6) Upload your working <netid>_part03_xor_nonce_reuse_skel.py code as a separate file.
(20 points)